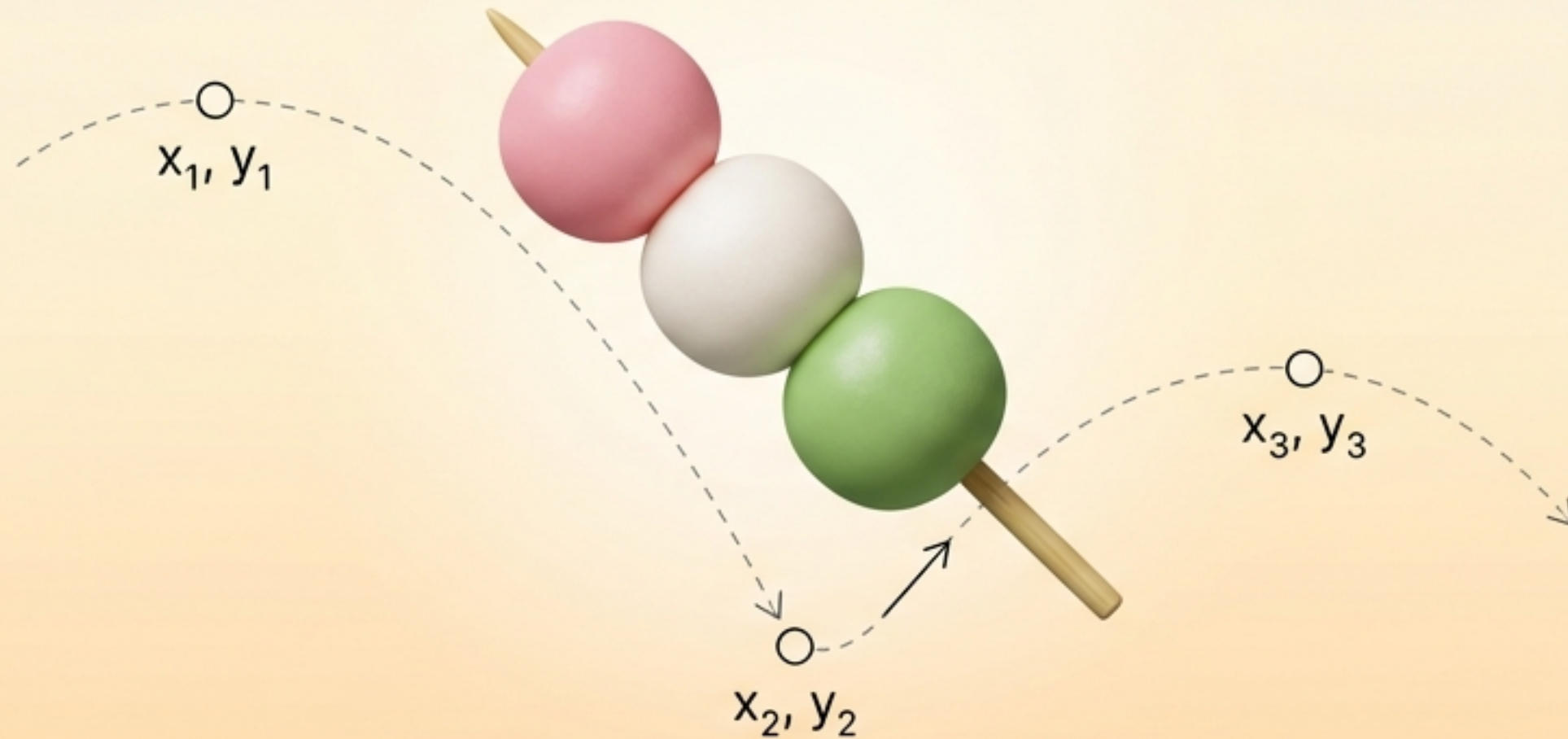
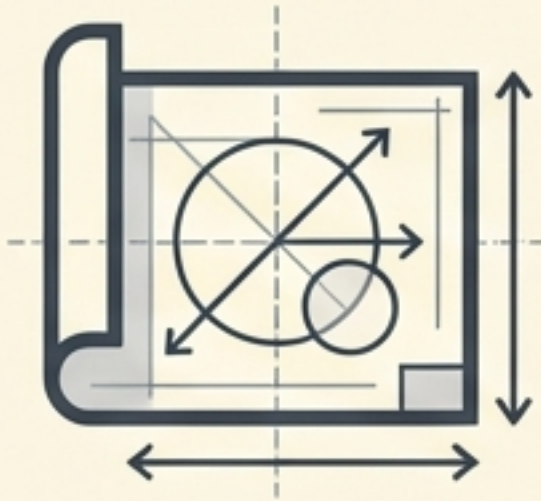


Anatomía de un mundo interactivo en Android



Descubriendo Kotlin, Físicas y Jetpack Compose
a través de una aplicación desde cero.

Una aplicación moderna divide sus responsabilidades en tres pilares



Mochi

El Qué. La estructura de la información.



MotorMochis

El Cómo. Las reglas físicas y matemáticas.



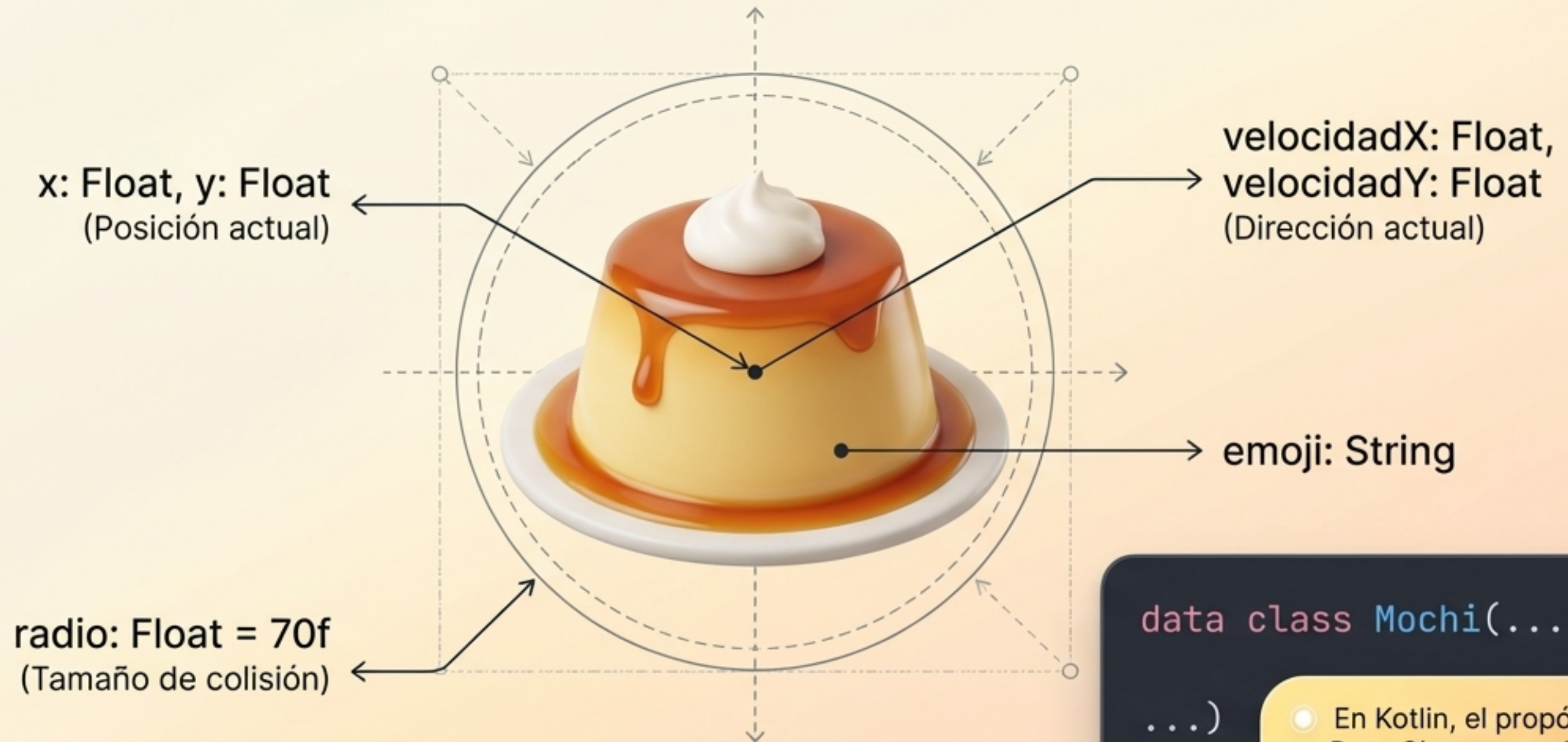
PantallaMochis

El Dónde. El lienzo visual interactivo.



Mezclar estas responsabilidades es el error número uno del principiante.

Las 'Data Classes' definen la identidad sin mezclar lógica compleja



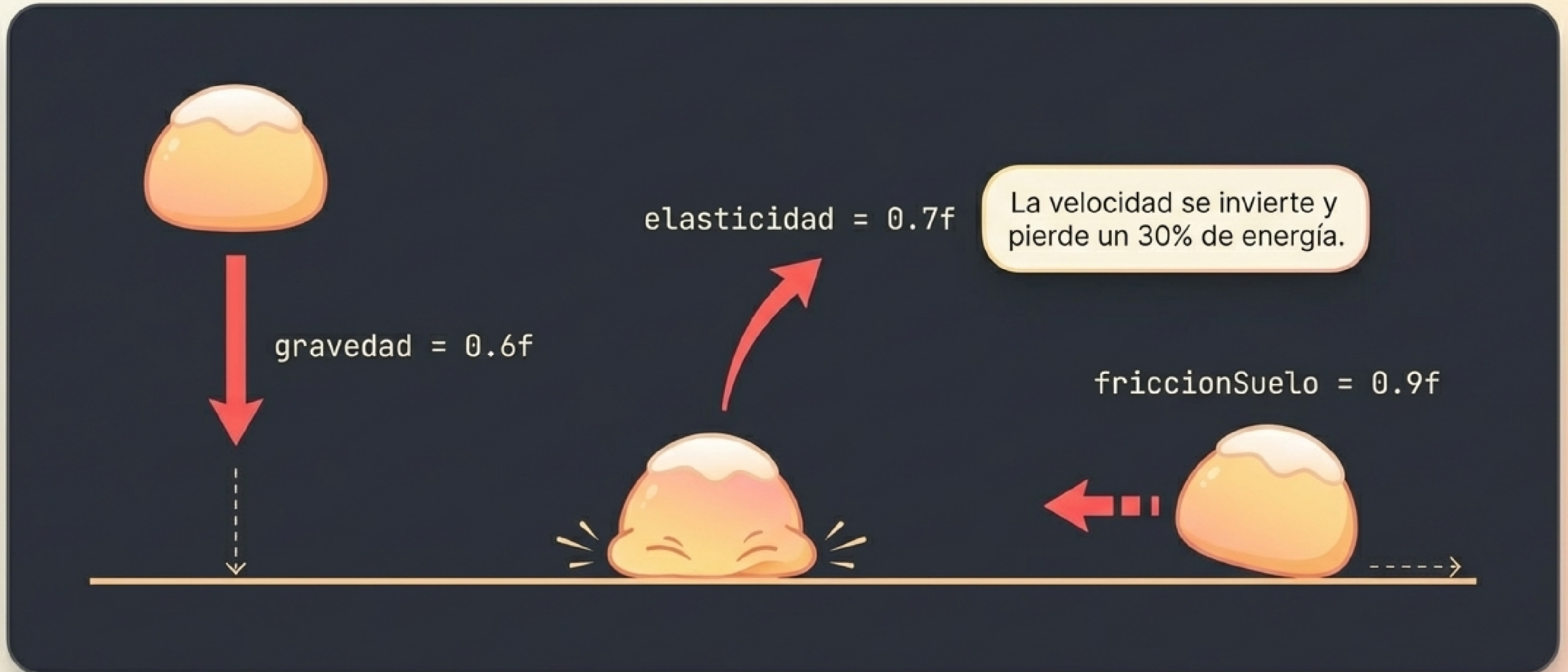
```
data class Mochi(...  
...)
```

- En Kotlin, el propósito de una Data Class es exclusivamente almacenar información pura.

El motor centraliza las matemáticas y aísla las reglas del universo



Variables sencillas simulan fuerzas físicas complejas



Evita los valores mágicos usando 'Companion Objects'

El camino novato

```
if (mochis.size > 30)
```



¿Por qué 30? Es un "número mágico" sin contexto.

La vía Pro Kotlin

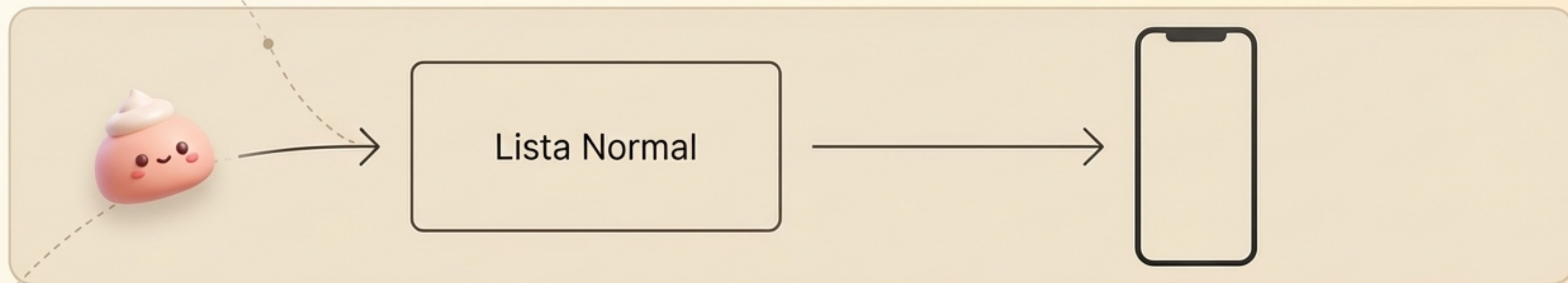
```
companion object {  
    const val MAX_MOCHIS = 30  
}
```



Define reglas universales una sola vez. Mejora la legibilidad y el mantenimiento.

Un Companion Object pertenece a la clase entera, no a un Mochi individual.

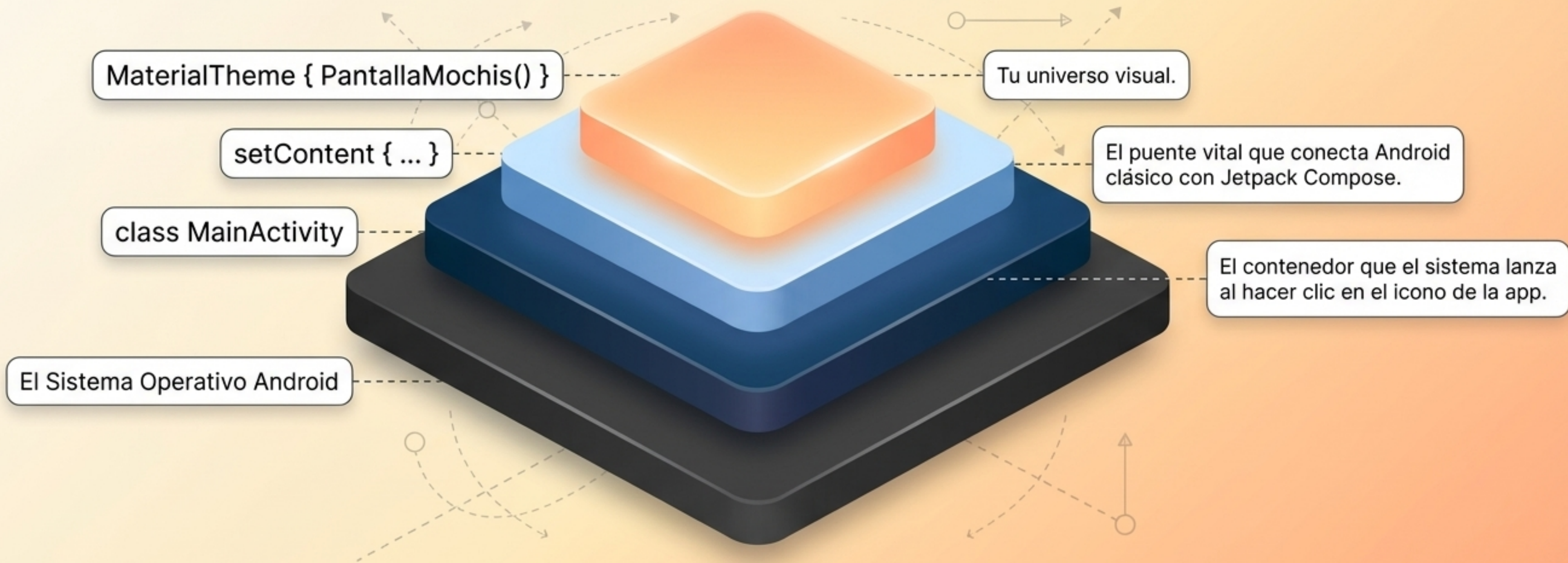
Las listas reactivas avisan a la pantalla cuando algo cambia



```
var mochis = mutableStateListOf<Mochi>()
```

Sin esta magia reactiva, el Motor calcularía las físicas perfectamente, pero el usuario vería una pantalla congelada.

Todo ecosistema Android nace dentro de una Actividad principal



Una aplicación no flota en el vacío; requiere este andamiaje para existir en el dispositivo.

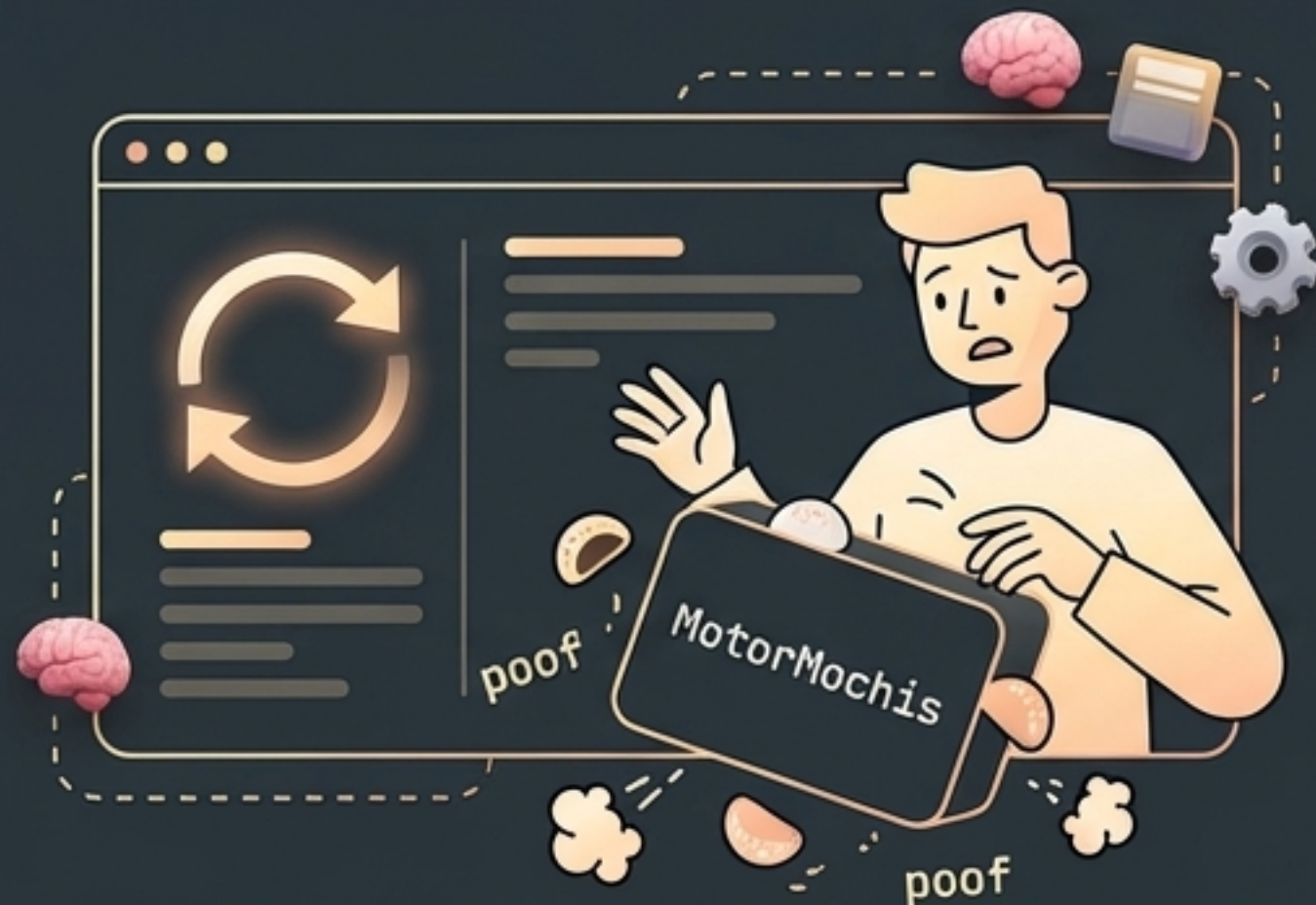
El diseño moderno dibuja fondos matemáticamente sin imágenes pesadas

```
Brush.verticalGradient(  
    Color(0xFFFFF3E0),  
    Color(0xFFFFCC80)  
)
```

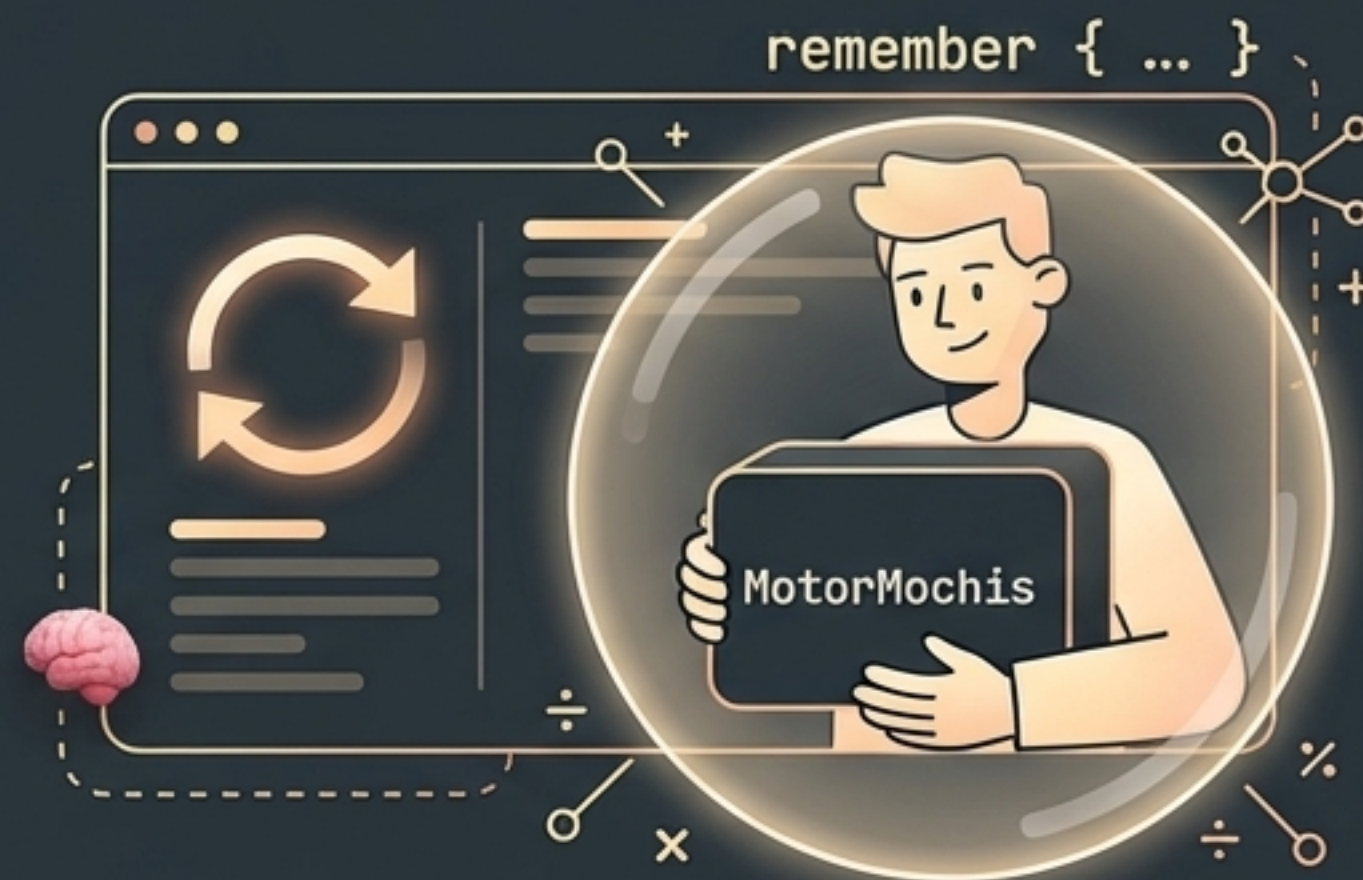


Renderizar matemáticamente consume menos memoria y se adapta a cualquier resolución de pantalla.

La función 'remember' protege la memoria durante los redibujos constantes



Sin remember: Compose olvida todo. Se crea un motor nuevo vacío ¡y los Mochis desaparecen 60 veces por segundo!



Con remember: Compose dice: "Tranquilo, ya guardé este Motor antes, continuemos".

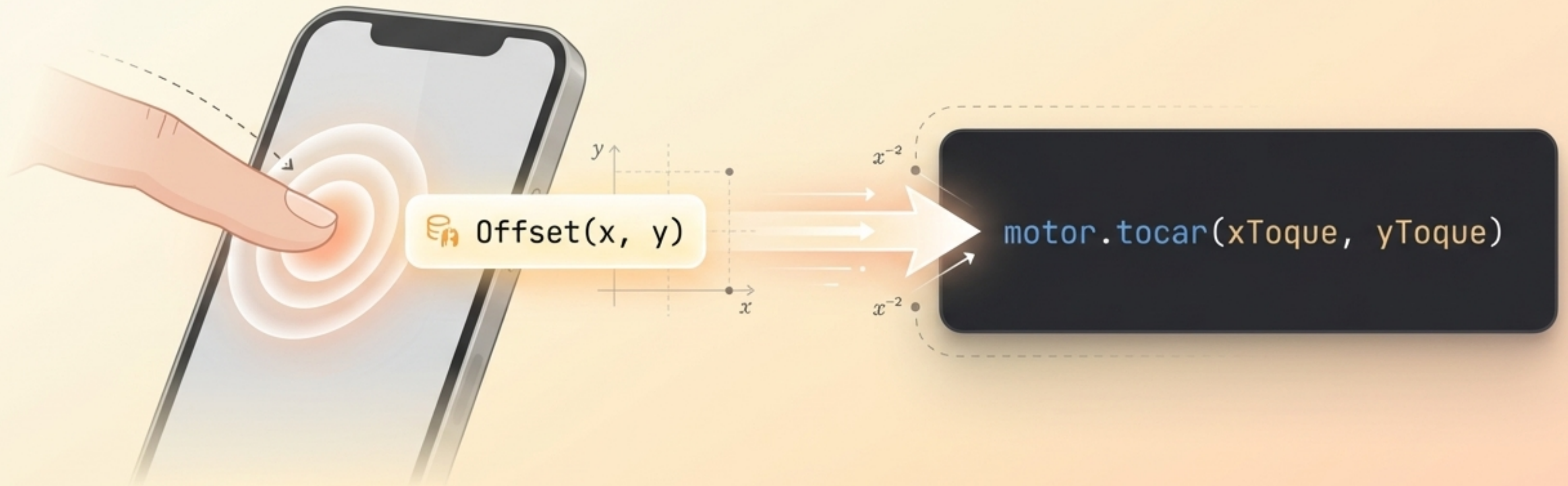
El componente Canvas ofrece control absoluto píxel por píxel

```
mochis.forEach {  
    drawText(mochi.emoji, mochi.x, mochi.y)  
}
```



El Canvas es un espacio en blanco de altísimo rendimiento. A diferencia de los botones estándar, aquí dictamos la posición exacta de cada elemento en el espacio bidimensional.

Convertimos toques físicos en coordenadas matemáticas exactas



El modificador **pointerInput** y **detectTapGestures** actúan como traductores: convierten el mundo físico analógico en el idioma digital del Motor.

La separación de responsabilidades garantiza un código escalable

MotorMochis
(Estado y Lógica)

Calcula la gravedad y colisiones

Lista pura de objetos matemáticos

Procesa las coordenadas de impacto

PantallaMochis
(Interfaz y Compose)

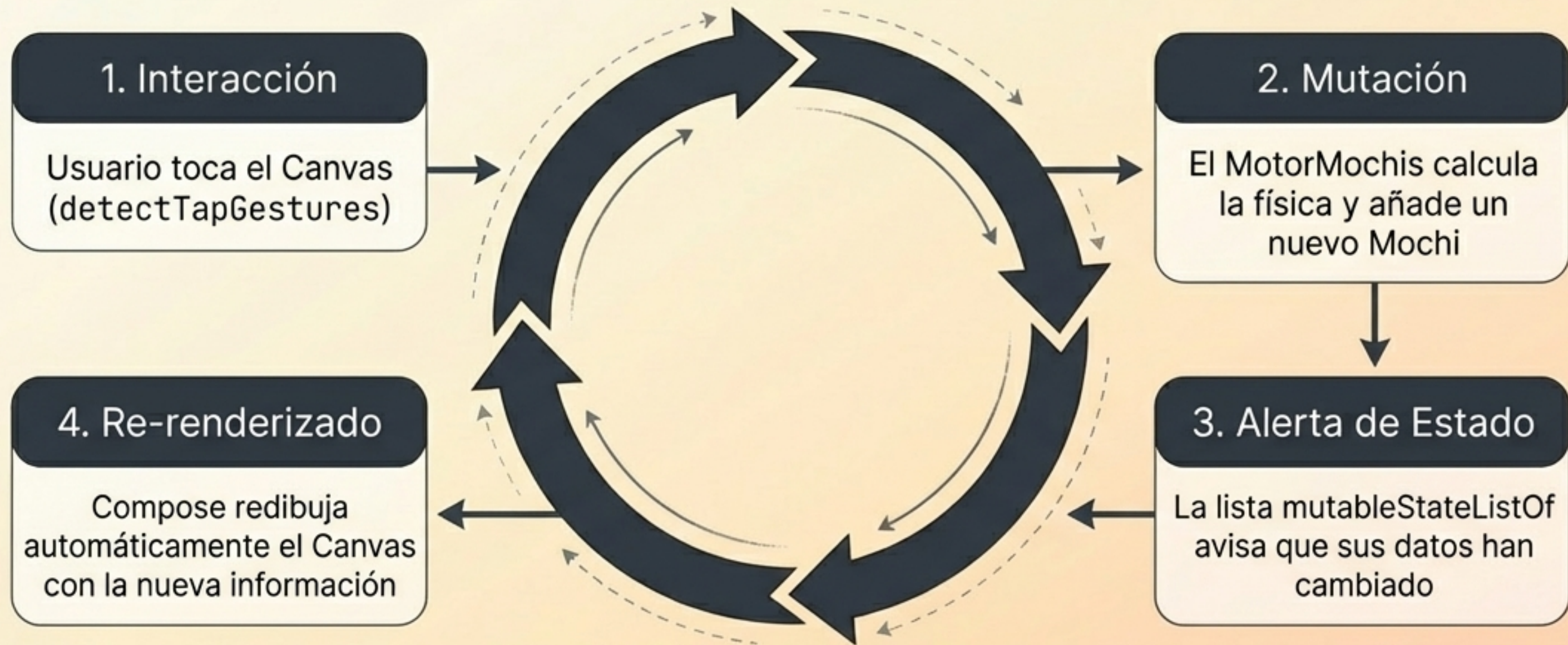
Dibuja el fondo Zen y los emojis

Lienzo de dibujo (Canvas) interactivo

Escucha el toque físico del dedo

Esta arquitectura permite cambiar el diseño visual mañana sin tocar una sola regla física.

Compose funciona como un bucle infinito de reacción unidireccional



Unidireccional: La pantalla nunca altera los datos directamente; solo reporta acciones al Motor y espera el nuevo estado.

Tu primer mundo interactivo está listo para evolucionar



- ✓ Has dominado la separación entre Estado y UI.
- ✓ Entiendes la reactividad de Compose.

Próximos desafíos:

- Añadir sonidos de rebote al impactar el suelo.
- Implementar rotación visual usando velocidadX.
- Investigar sobre ViewModel (La evolución profesional del Motor).